C Programming Notes for Unit-1

History and Importance of 'C'

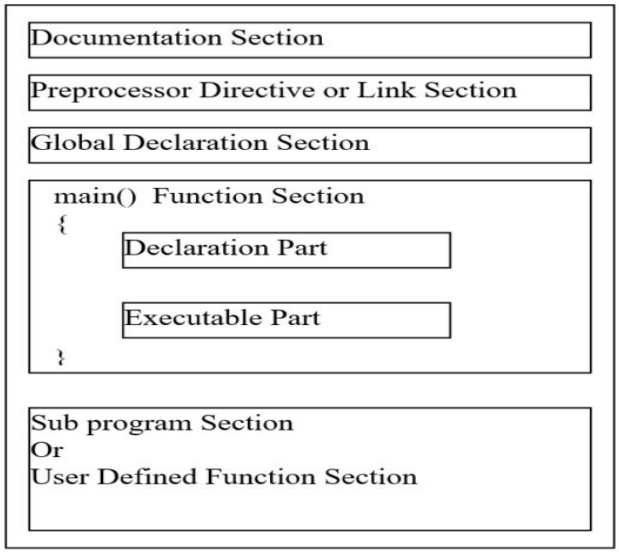
- C was originally developed in the 1970s, by Dennis Ritchie at Bell Telephone Laboratories, Inc.
- C is a High level , general –purpose structured programming language. Instructions of C consists of terms that are very closely same to algebraic expressions, consisting of certain English keywords such as if, else, for ,do and while
- C contains certain additional features that allows it to be used at a lower level , acting as bridge between machine language and the high level languages.
- This allows C to be used for system programming as well as for applications programming
- C is called as a robust language, which has so many built-in functions and operations, which can be used to write any complex program.
- The C programming language is used to create operating systems, compilers, interpreters, third-party libraries, and databases.
- 'C' Programs are efficient and fast.
- 'C' language is best for structured programming, where the user can think of a problem in terms of function modules (or) blocks.
- It has the ability to extend itself.
- C is a general-purpose structured language that promotes code organization through functions.

Applications of 'C'

- **Operating Systems:** Because of its low-level control and performance, C as a language is a goto for operating system development.
- **Embedded Systems:** C is heavily used in embedded systems programming, in large part due to its ability to interact directly with hardware.
- **Compilers and Interpreters:** Because of its efficiency, C is frequently used in developing compiler and interpreters.
- **Databases:** C is employed in database systems for performance-critical components.
- **Networking and Communications:** Its low-level socket programming capabilities enable efficient communication in distributed systems.
- **Game Development:**

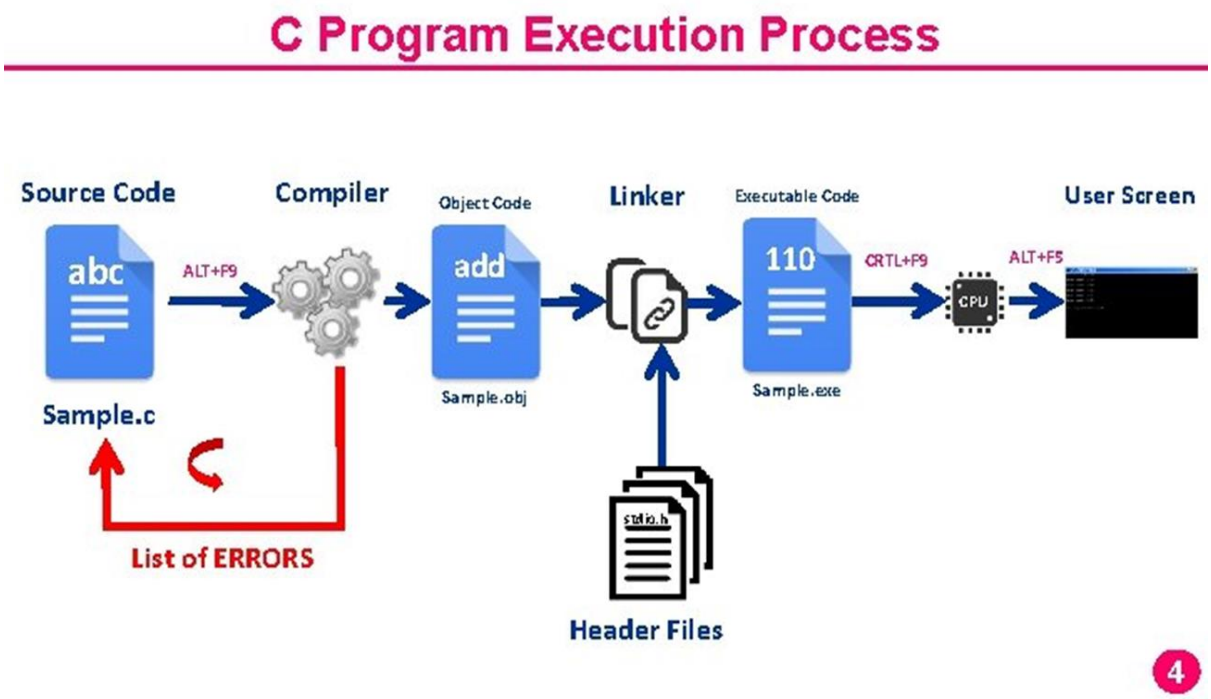
- C is often employed in game development for performance-heavy tasks like rendering and physics calculations.
- **Scientific Computing:** C supports high-performance scientific applications, including numerical simulations and supercomputing tasks.
- **Real-Time Systems:** C is essential for real-time systems, including aerospace control (usually flight controllers), medical devices, and industrial automation

Basic Structure of 'C' Program



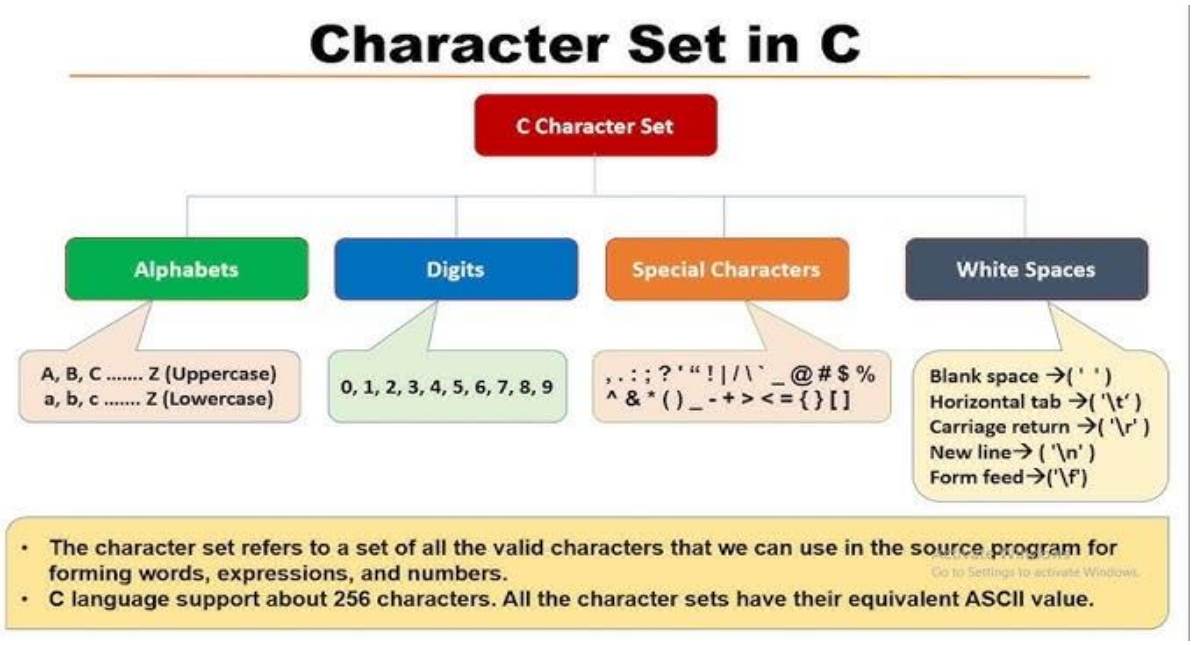
Example Program

```
/*program to print a message*/

#include<stdio.h>

main()

{

        printf("This is C program\n");

}
```



Lecture-2

Character Set in C



Tokens in C

A token is the smallest element of a program that is meaningful to the compiler. Tokens can be classified as follows:

- Keywords
- Identifiers
- Constants
- Strings
- Special symbols
- Operators

Keyword

Keywords are pre-defined or reserved words in a programming language. Each keyword is meant to perform a specific function in a program. Since keywords are referred names for a compiler, they can't be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed. You cannot redefine keywords. C language supports 32 keywords which are given below:

Here's a list of the 32 keywords in C:

- Data Types: auto, char, double, float, int, long, short, signed, unsigned, void, enum, struct, union, typedef

- Control Flow: if, else, for, while, do, switch, case, default, break, continue, goto, return
- Storage Class: auto, extern, register, static
- Others: const, sizeof, volatile, _Packed

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	Volatile
const	float	short	Unsigned

Identifiers

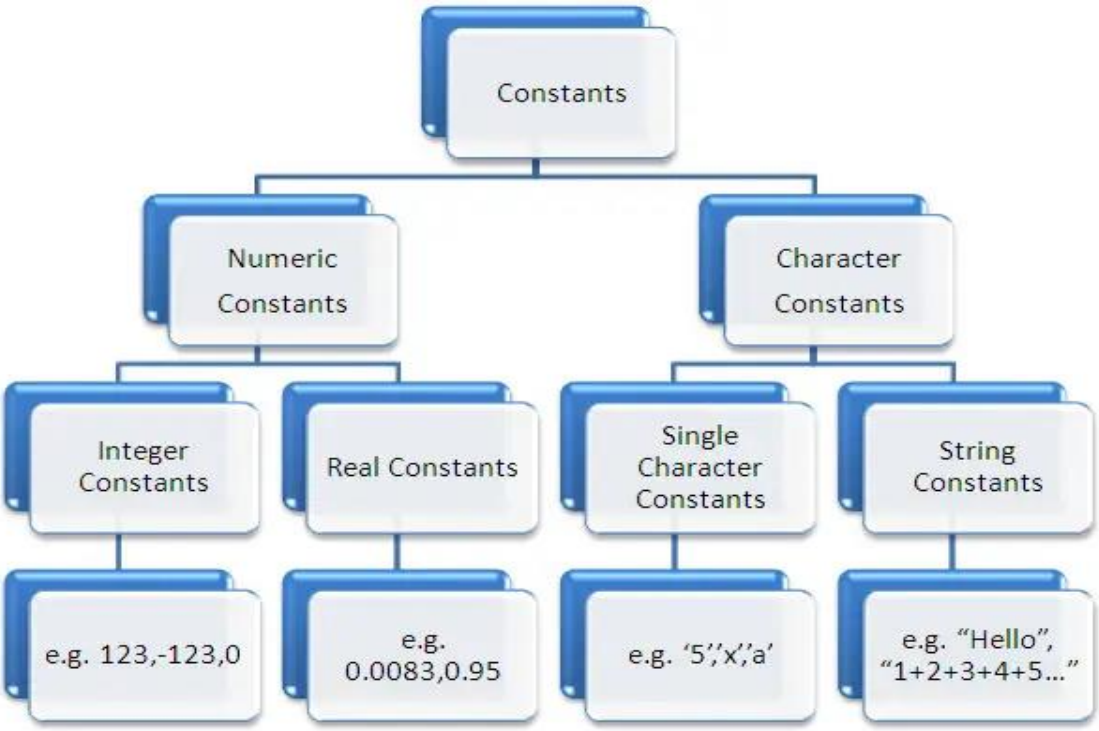
Identifiers are used as the general terminology for naming of variables, functions and arrays. These are user defined names consisting of arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character. Identifier names must differ in spelling and case from any keywords. You cannot use keywords as identifiers; they are reserved for special use. Once declared, you can use the identifier in later program statements to refer to the associated value. A special kind of identifier, called a statement label, can be used in goto statements.

Rules for naming c identifiers

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.
- It must not contain white space.
- It should be up to 31 characters long as only first 31 characters are significant.

Constants

Constants in C refer to fixed values that do not change during the execution of a program. Several types of constants supported in C are shown in the chart below



Strings

Strings are nothing but an array of characters ended with a null character ('\0').This null character indicates the end of the string. Strings are always enclosed in double quotes. Whereas, a character is enclosed in single quotes in C.

- char string[15] = {'p', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g','\0'};
- char string[20] = "programming";
- char string [] = "programming";

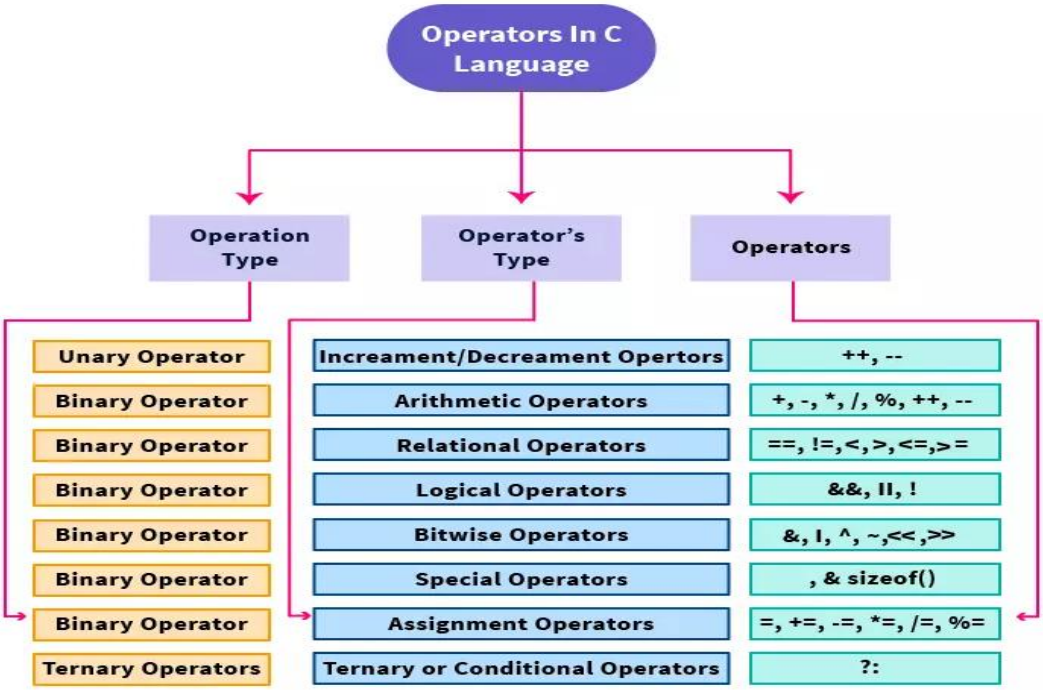
Special Symbols/Characters

Special Characters

Character	Name		Character	Name
,	Comma		&	Ampersand
.	Period		^	Caret
;	Semicolon		*	Asterisk
:	Colon		-	Minus
?	Question mark		+	Plus
'	Single quote		=	Equal
"	Double quote		<	Less than
!	Exclamation		>	Greater than
|	Vertical bar		(	Left parenthesis
/	Slash		)	Right parenthesis
\	Back slash		[	Left square bracket
~	Tilde		]	Right square bracket
_	Underscore		{	Left curly bracket
$	Dollar		}	Right curly bracket
%	Percentage		#	Hash-Number sign

Operators

An operator is a symbol that tells the computer to perform certain mathematical or logical manipulation on data stored in variables. The variables that are operated are termed as operands. C operators can be classified into 8 types. These operators are given below.



Lecture – 3

Introduction to Algorithm and Flowchart

Algorithm:

In programming, algorithm is a set of well-defined instructions in sequence to solve the problem.

Algorithm to add 2 Numbers

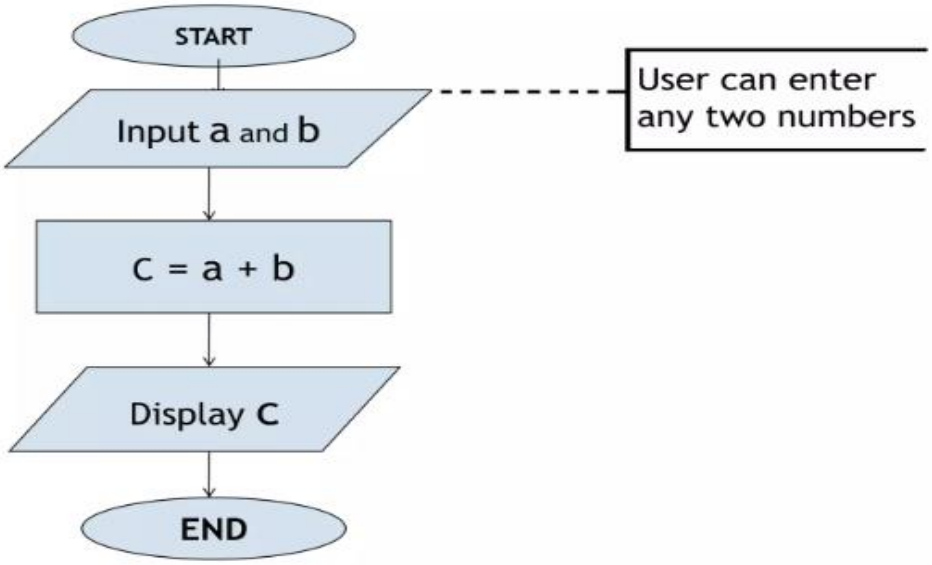
Step 1: Start
Step 2: Enter two numbers in a and b
Step 3: Add a and b and store in c
Step 4: Display c
Step 5: End

Flowchart:

A flowchart is a diagrammatic representation of an algorithm. A flowchart can be helpful for both writing programs and explaining the program to others.

Symbol	Function	Description
	Start/End	Start and end point
	Input/output	Input and output operations
	Processing operation	Editing and calculation of data
	Decision	Check logical condition
	Connector	Indicates logical flow from one page to another
	Direction of logic	Direction of flow of logic
	Comment	Indicates any comments for explanatory notes

Flowchart to add 2 Numbers

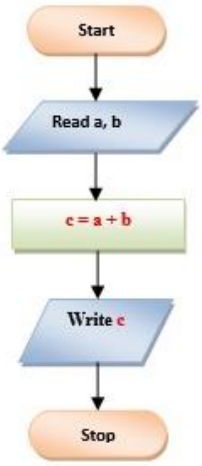


To find sum of two numbers

Algorithm

1. Start
2. Read a, b
3. c = a + b
4. Print or display c
5. Stop

Flowchart



Program

```
#include<stdio.h>

int main()
{
    int a, b, c;

    printf("Enter value of a: ");
    scanf("%d", &a);

    printf("Enter value of b: ");
    scanf("%d", &b);
    c = a+b;

    printf("Sum of given two numbers is: %d", c);

return 0;
}
```

Constants

In C programming, constants are fixed values that remain unchanged throughout the execution of a program. Unlike variables, which can have their values modified during runtime, constants hold a single, immutable value once defined

There are several types of constants in C:

- Integer Constants:

Whole numbers without fractional parts. They can be decimal, octal (prefixed with 0), or hexadecimal (prefixed with 0x).

Example: 10, 012, 0xA

- Floating-Point Constants:

  Numbers with fractional parts, represented with a decimal point or in exponential form (using 'e' or 'E').

  Example: 3.14, 1.2e-5

- Character Constants:

  Single characters enclosed in single quotes.

  Example: 'A', 'c', '\n' (newline character)

- String Literals:

  Sequences of characters enclosed in double quotes. They are treated as arrays of characters terminated by a null character (\0).

  Example: "Hello, World!"

- Enumeration Constants:

  Defined using the enum keyword, providing a way to assign symbolic names to integer values.

  Example: enum days {MON, TUE, WED};

How to define a constant?

Defining Constants

There are two simple ways in C to define constants:

1. Using **#define** preprocessor.
2. Using **const** keyword.

- **Using the #define preprocessor directive**: This creates symbolic constants that are replaced by their values during the pre-processing stage.

```
#define MAX_BUFFER_SIZE 1024
#define GREETING "Welcome!“
```

-

- **Using the const keyword:** This is the preferred and type-safe method. You declare a variable as const to make its value unchangeable after initialization.

```
const int MAX_USERS = 100;
const float PI = 3.14159;
```

Constants are crucial for making code more readable, maintainable, and less prone to errors by preventing accidental modification of important values.

Example Program Using #define preprocessor.

```
#include <stdio.h>
#define LENGTH 10
#define WIDTH  5
#define NEWLINE '\n'
void main()
{
  int area;
  area = LENGTH * WIDTH;
  printf("value of area : %d", area);
  printf("%c", NEWLINE);
  }
```

When the above code is compiled and executed, it produces following result:

value of area : 50

Example Program Using const Keyword

```
#include <stdio.h>
void main()
{
  const int  LENGTH = 10;
  const int  WIDTH  = 5;
  const char NEWLINE = '\n';
  int area;
  area = LENGTH * WIDTH;
  printf("value of area : %d", area);
  printf("%c", NEWLINE);
 }
```

Lecture - 4

Datatypes in C

In C programming, data types are used to define the type of data a variable can hold. They tell the compiler how much memory to allocate and how to interpret the bits stored in that memory.

1. Primary (Fundamental) Data Types
2. Derived Data Types
3. Enumeration (Enum)
4. User-defined Data Types

1. **Primary (Fundamental) Data Types**

Type	Size (Typical)	Description
int	4 bytes	Integer numbers (e.g., 10, -5)
float	4 bytes	Single-precision decimal numbers
double	8 bytes	Double-precision decimal numbers
char	1 bytes	Single character (e.g., 'A')
void	0 byte	No value (used for functions)

Example:

```
int i, j, k;  [ 3 variables -  i, j, k of integer type ]
char  c, ch;  [ 2 variables –  c, ch of character type ]
float f, salary;  [variables –  f, salary of floating type ]
double d; [ 1 variable –  d of double type ]
```

The void Type

The void type specifies that no value is available. It is used in three kinds of situations:

	Types and Description
1.	**Return type of a function as void:** There are various functions in C who do not return a value or you can say they return void. A function with no return value has the return type as void. For example, void fun (int a);
2.	**Function arguments as void:** There are various functions in C who do not accept any parameter. A function with no parameter can accept as a void. For example, int rand(void);
3.	**Pointers to void:** A pointer of type void * represents the address of an object, but not its type. For example a memory allocation function void *malloc (size_t size); returns a pointer to void which can be casted to any data type.

sizeof operator

To get the exact size of a type or a variable on a particular platform, the sizeof operator. The expressions *sizeof(type)* yields the storage size of the object or type in bytes.

Example to get the size of int type on any machine:

```
#include <stdio.h>

void main()

{

  printf("Storage size for int : %d \n", sizeof(int))

}
```

2. **Derived Data Types**
   These are based on fundamental types:
- Array – A collection of elements of the same type
  Example: int arr[10];
- Pointer – Stores the address of another variable
  Example: int *ptr;
- Function – A block of code with a return type and parameters
  Example: int add(int, int);

3. **Enumeration (enum)**
   Used to define a variable that can hold a set of predefined constants.

   enum week { Mon, Tue, Wed, Thu, Fri };

4. **User-defined Data Types**

- **Structure (struct)** – Groups variables of different types
  struct Person { char name[20]; int age; };
- **Union (union)** – Similar to structure, but shares memory for all members
  union Data { int i; float f; char str[20]; };
- **Typedef** – Used to create alias for data types
  typedef unsigned int uint;

Lecture-5

1. **Input and Output Statements in C**
   C uses standard I/O library functions declared in the <stdio.h> header file.

- Input: Getting data from the user (keyboard)
- Output:
- Sending data to the screen (console)

  Include this at the top of your code:
  #include <stdio.h>

2. **Reading a Character (Input)**

   Use getchar() or scanf():
   getchar()
   Reads a single character from input.
   char ch; ch = getchar();

3. **Writing a Character (Output)**

   Use putchar() or printf():
   putchar()
   Writes a single character to output.
   char ch = 'A';
   putchar(ch);

4. **Formatted Input and Output Statements**

   scanf() – Formatted Input
   Used to read formatted input (e.g., integers, floats, strings):
   int a; float b; char name[20];
   scanf("%d %f %s", &a, &b, name);

Common format specifiers:

printf() – Formatted Output

Used to print values in formatted style:

printf("Integer: %d, Float: %.2f, Name: %s", a, b, name);

Format	Type
%d	Integer
%f	Float
%c	Character
%s	String
%lf	Double

Example Program

```
#include <stdio.h>
int main()
{
   char ch;
   int num;
   float f;
   printf("Enter a character: ");
   ch = getchar();  // Reading a character
   printf("Enter an integer and a float: ");
   scanf("%d %f", &num, &f);  // Formatted input
   printf("You entered:\n");
   putchar(ch);            // Writing a character
   printf("\nInteger: %d\nFloat: %.2f\n", num, f);  // Formatted output
   return 0;
}
```

Some Programs for Practice

1. C Program to Print an Integer Entered By the User

```
#include <stdio.h>
int main() {
 int num;
 printf("Enter an integer: ");
 scanf("%d", &num);
 printf("You entered: %d\n", num);
 return 0;
}
```

2. C Program to Multiply two Floating-Point Numbers

```
#include <stdio.h>
int main() {
float num1, num2, product;
printf("Enter two floating-point numbers: ");
scanf("%f %f", &num1, &num2);
product = num1 * num2;
printf("Product: %f\n", product);
 return 0;
}
```

3. C Program to Print the ASCII Value of a Character

```
#include <stdio.h>
int main() {
 char ch;
 printf("Enter a character: ");
 scanf("%c", &ch);
 printf("ASCII value of %c = %d\n", ch, ch);
```

```
 return 0;
}
```

4. C Program to Calculate Fahrenheit to Celsius

```
#include <stdio.h>
int main() {
 float fahrenheit, celsius;
 printf("Enter temperature in Fahrenheit: ");
 scanf("%f", &fahrenheit);
 celsius = (fahrenheit – 32) * 5 / 9;
 printf("Temperature in Celsius: %f\n", celsius);
 return 0;
}
```

5. C Program to Find the Size of int, float, double, and char

```
#include <stdio.h>
int main() {
 printf("Size of int: %d bytes\n", sizeof(int));
 printf("Size of float: %d bytes\n", sizeof(float));
 printf("Size of double: %d bytes\n", sizeof(double));
 printf("Size of char: %d byte\n", sizeof(char));
 return 0;
}
```